

Sistema de Controle Acadêmico ClassroomPB

Relatorio-release1



Release 1 — Núcleo Acadêmico

Componentes da Equipe: Alessia Bianca, Arthur Freire, Keison Raniery, José Rafael

Objetivo da Entrega: Apresentar a especificação técnica e a base estrutural do sistema, englobando os requisitos de cadastro, autenticação e gestão acadêmica básica (RF01 a RF14).

Sumário

1. Descrição da Arquitetura Utilizada.....	3
Camadas Estruturais:.....	3
• Fluxo 1: Autenticação de Usuário (Login).....	4
• Fluxo 2: Ativação de Período Letivo.....	5
3. Padrões de Projeto Aplicados.....	6
Factory Method (Padrão Creacional).....	6
Singleton (Padrão Estrutural / Gerenciamento de Estado).....	6
Facade / Fachada (Padrão Estrutural).....	7
4. Funcionalidades Desenvolvidas e Evidências.....	7
US01 — Cadastro de Usuários no Sistema e Bloqueio de Duplicidade.....	7
US02 — Login no Sistema e Validação de Perfil.....	7
US03 — Cadastro de Cursos no Sistema e Bloqueio de Código Duplicado.....	8
US04 — Cadastro de Disciplinas e Vinculação de Pré-requisitos Opcionais.....	8
US05 — Cadastro e Controle de Estados do Período Letivo.....	9
US06 — Oferta de Turmas para Disciplinas em Períodos Ativos.....	9
US09 — Controle de Ativação e Encerramento de Período Letivo.....	9
US10 — Oferta de Turmas para Disciplinas pelo Coordenador.....	10
US11 — Definição de Atributos, Restrições e Responsáveis da Turma.....	10
US12 — Validação de Conflito de Horário de Professor.....	10
US13 — Obrigatoriedade de Professor Responsável na Turma.....	11
5. Relatório de Testes Automatizados e Cobertura.....	12
Cenários Cobertos:.....	12

1. Descrição da Arquitetura Utilizada

O sistema ClassroomPB foi desenvolvido seguindo uma variação da arquitetura em camadas com separação clara de responsabilidades, o que garante a manutenibilidade e a testabilidade do ecossistema de software. A arquitetura divide-se entre componentes de apresentação textual, controladores de regras de negócio, fábricas de inst. instanciação e persistência local.

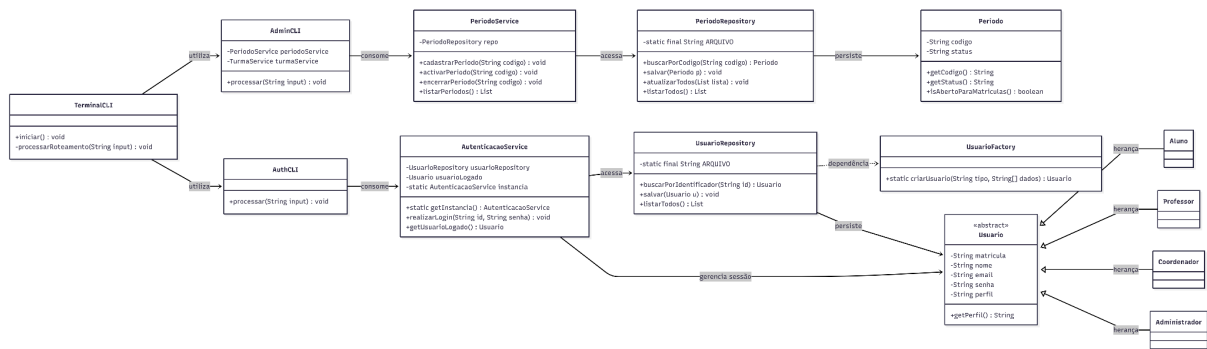
Camadas Estruturais:

- Apresentação (`br.edu.uepb.classroompb.view`): Contém os componentes CLI (`TerminalCLI`, `AuthCLI`, `AdminCLI`, `AlunoCLI`, `CoordenadorCLI`, `ProfessorCLI`) encarregados de efetuar o *parsing* dos comandos textuais e interagir com o usuário.
- Regras de Negócio / Serviço (`br.edu.uepb.classroompb.service`): Camada central (`AutenticacaoService`, `TurmaService`) onde as políticas do sistema são validadas de forma isolada de interfaces ou persistência física.
- Padrões de Criação (`br.edu.uepb.classroompb.factory`): Utiliza o padrão *Factory Method* (`UsuarioFactory`) para encapsular e polimorfar a instanciação de perfis do sistema de forma desacoplada.
- Persistência (`br.edu.uepb.classroompb.repository`): Gerenciadores de dados (`UsuarioRepository`, `TurmaRepository`) responsáveis pela leitura e escrita em arquivos binários de objetos serializados (`usuarios.dat`), conferindo persistência mesmo após o encerramento do processo

2. Modelagem e Arquitetura do Sistema

2.1 Diagrama de Classes UML

O Diagrama de Classes ilustra o forte desacoplamento obtido através da arquitetura em camadas. A classe `AutenticacaoService` implementa o padrão *Singleton*, mantendo uma referência única em memória para salvaguardar a sessão do `Usuario` logado. As classes herdeiras de `Usuario` (como `Aluno` e `Coordenador`) são instanciadas de forma polimórfica pela `UsuarioFactory`, blindando os serviços contra o acoplamento direto de instanciação concreta. A persistência é isolada nos componentes `Repository`, operando sobre arquivos de dados locais de maneira transparente para a camada de visualização.

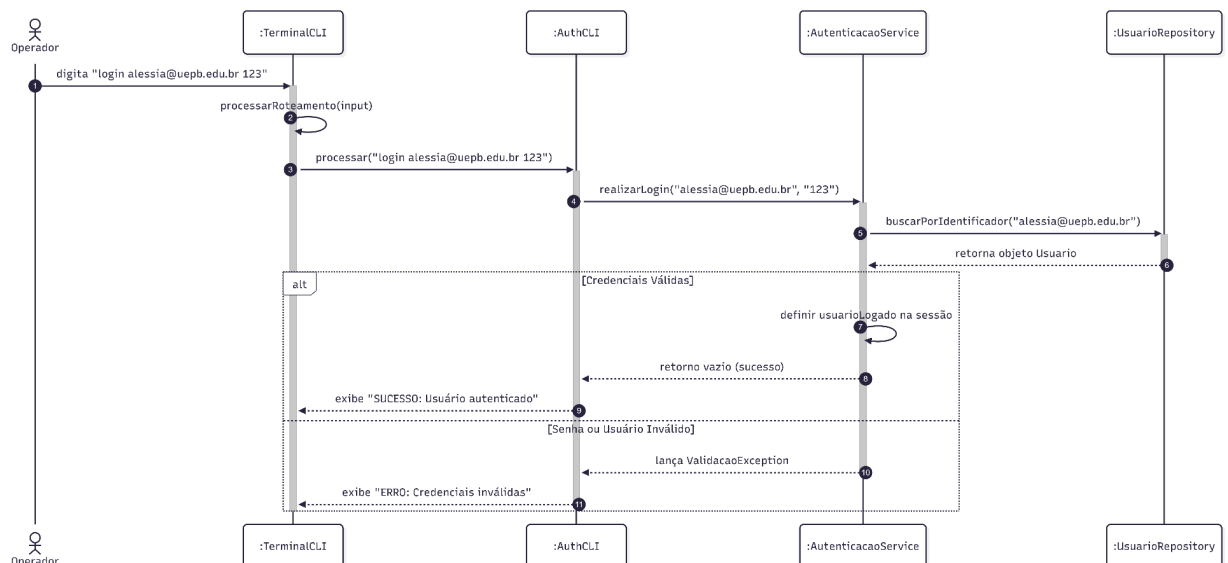


2.2 Diagramas de Sequência:

Os diagramas abaixo demonstram o comportamento dinâmico e a ordem temporal de troca de mensagens do ecossistema de software.

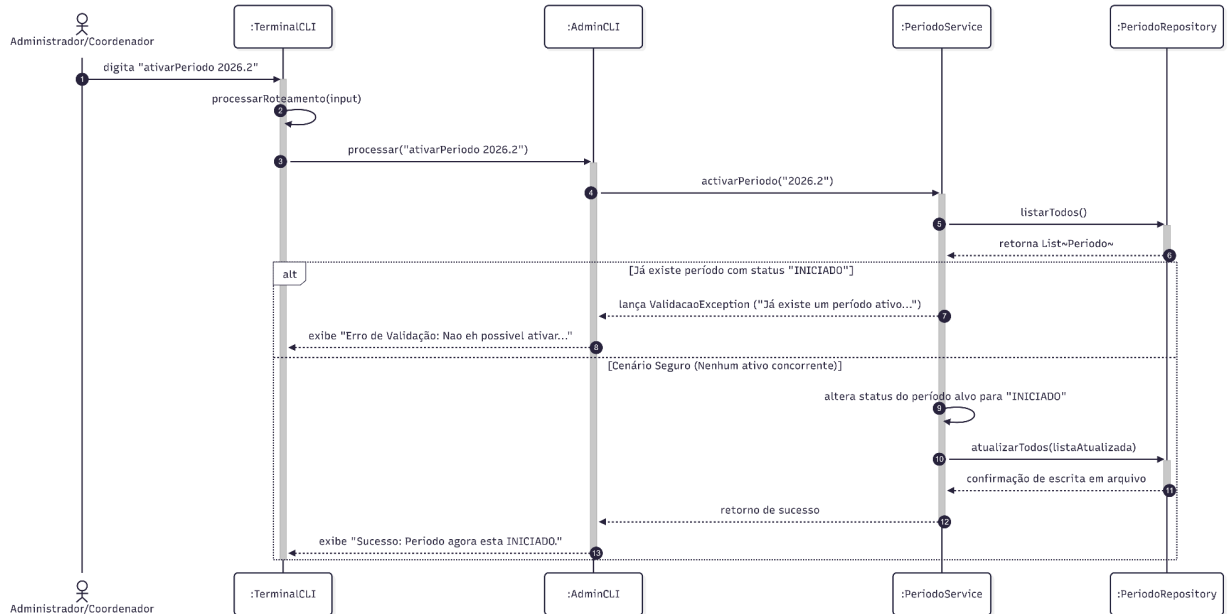
● Fluxo 1: Autenticação de Usuário (Login)

Mapeia o ciclo percorrido desde a captura da string de texto até o estabelecimento da sessão no sistema.



- **Fluxo 2: Ativação de Período Letivo**

Mapeia a inteligência de concorrência cronológica restritiva desenvolvida na Task 1901.



3. Padrões de Projeto Aplicados

Para garantir o desacoplamento, a testabilidade e a manutenibilidade do sistema ClassroomPB, a arquitetura foi fundamentada na aplicação estratégica de padrões de projeto (*Design Patterns*) consagrados pela engenharia de software. Durante a primeira release, destacam-se a implementação dos seguintes padrões:

Factory Method (Padrão Creacional)

- Onde foi aplicado: Na inicialização e diferenciação dos perfis de usuários do sistema (**UsuarioFactory**).
- Justificativa Técnica: No modelo original, o serviço de autenticação precisava conhecer as classes concretas (**Aluno**, **Professor**, **Coordenador**, **Administrador**) e utilizar operadores condicionais **new** para instanciá-las. Com a introdução do *Factory Method*, o acoplamento direto foi removido. O **AutenticacaoService** agora interage apenas com a abstração **Usuario**, delegando a responsabilidade de criação polimórfica para a fábrica encapsulada. Isso facilita a futura extensão do sistema caso novos tipos de perfis (como *Diretor* ou *Técnico*) sejam adicionados, sem a necessidade de modificar as regras de negócio existentes.

Singleton (Padrão Estrutural / Gerenciamento de Estado)

- Onde foi aplicado: No gerenciamento do estado de sessão unificada dentro da classe `AutenticacaoService`.
- Justificativa Técnica: O controle de quem está logado no sistema precisa ser centralizado e acessível por diferentes componentes da CLI (como a `CoordenadorCLI` e a `AuthCLI`) para validação de barreiras de segurança. A aplicação do padrão *Singleton* garante que exista apenas uma instância ativa do serviço de autenticação na memória durante todo o ciclo de vida do processo, encapsulando com segurança o ponteiro global do `usuarioLogado`.

Facade / Fachada (Padrão Estrutural)

- Onde foi aplicado: Na interface de roteamento central representada pela classe `TerminalCLI`.
- Justificativa Técnica: As regras de negócio e os repositórios locais do sistema possuem subsistemas complexos de arquivos e validações (`TurmaService`, `PeriodoService`, `UsuarioRepository`, etc.). Para evitar que o método principal (`Main.java`) gerencie essa complexidade diretamente, a `TerminalCLI` atua como uma *Fachada* simplificada. Ela expõe um único método de inicialização (`iniciar()`) e resolve internamente o parsing e o roteamento de subsistemas delegando o processamento para as sub-interfaces (`AdminCLI`, `CoordenadorCLI`, `AuthCLI`), mascarando a complexidade estrutural por trás de comandos de texto simples.

4. Funcionalidades Desenvolvidas e Evidências

US01 — Cadastro de Usuários no Sistema e Bloqueio de Duplicidade

- **Descrição Técnica:** Implementação do mecanismo de cadastro polimórfico de perfis de usuários utilizando o padrão de projeto *Factory Method* (`UsuarioFactory`). O sistema realiza a validação de integridade de dados na camada de persistência (`UsuarioRepository`), impedindo de forma estrita o salvamento de duas contas contendo a mesma matrícula ou o mesmo e-mail na base de dados local (`usuarios.dat`).

```

=====
                ClassroomPB CLI
        Status: Modo Visitante (Não Autenticado)
=====
1. Realizar Login
2. Cadastrar Aluno
3. Cadastrar Professor
4. Cadastrar Coordenador
5. Cadastrar Administrador
0. Sair do Sistema
=====
Escolha uma opção: 5
Nome Completo (Use_Underlines_No_Lugar_De_Espacos): Keison_raniery
CPF: 10353465402
Matrícula: 0001
E-mail: keison@gmail.com
Senha: 1313
[Conflito de Cadastro] Erro: Usuário com a matrícula '0001' já está cadastrado.

```

(Matrículas iguais o sistema barra)

US02 — Login no Sistema e Validação de Perfil

- Descrição Técnica:** Construção do motor de autenticação unificado e dinâmico na camada de serviço (**AutenticacaoService**). O sistema permite que o usuário realize o login informando seu e-mail ou sua matrícula indistintamente. Ao validar as credenciais, o estado da sessão ativa é mantido globalmente por meio de um objeto unificado (**usuarioLogado**). A camada de apresentação (**AuthCLI**) atua como interceptadora, validando o nível de privilégio do perfil logado para bloquear comandos restritos em tempo de execução.

```

=====
                ClassroomPB CLI
        Status: Modo Visitante (Não Autenticado)
=====
1. Realizar Login
2. Cadastrar Aluno
3. Cadastrar Professor
4. Cadastrar Coordenador
5. Cadastrar Administrador
0. Sair do Sistema
=====
Escolha uma opção: 1
Digite o Identificador (E-mail/Matrícula): 0001
Digite a Senha: 1313
Acesso autorizado! Bem-vindo(a), perfil: ADMINISTRADOR
Sucesso: Login realizado com sucesso! Sessão ativa para o usuário.

```

(Validação de login com matrícula ou email e a senha)


```

=====
ClassRoomPB CLI
Status: Modo Visitante (Não Autenticado)
=====
1. Realizar Login
2. Cadastrar Aluno
3. Cadastrar Professor
4. Cadastrar Coordenador
5. Cadastrar Administrador
0. Sair do Sistema
=====
Escolha uma opção: 1
Digite o Identificador (E-mail/Matrícula): 0001
Digite a Senha: 1515
Erro: Credenciais inválidas (Usuário não encontrado ou senha incorreta).

```

```

=====
ClassRoomPB CLI
Status: Modo Visitante (Não Autenticado)
=====
1. Realizar Login
2. Cadastrar Aluno
3. Cadastrar Professor
4. Cadastrar Coordenador
5. Cadastrar Administrador
0. Sair do Sistema
=====
Escolha uma opção: 1
Digite o Identificador (E-mail/Matrícula): 0001
Digite a Senha: 1515
Erro: Credenciais inválidas (Usuário não encontrado ou senha incorreta).

```

```

=====
ClassRoomPB CLI
Usuário: Keison_Raniery [ADMINISTRADOR]
=====
1. Cadastrar Período Letivo
2. Ativar/Iniciar Período Letivo
3. Encerrar Período Letivo
4. Cadastrar Novo Curso
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: █

```



```

=====
                ClassroomPB CLI
    Usuário: Keison_Raniery [ADMINISTRADOR]
=====
1. Cadastrar Período Letivo
2. Ativar/Iniciar Período Letivo
3. Encerrar Período Letivo
4. Cadastrar Novo Curso
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: █

```

```

=====
                ClassroomPB CLI
    Usuário: alessia_bianca [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma (US10/US11)
3. Editar Turma Existente (US13/US14)
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: █

```

Java: Ready

```

=====
                ClassroomPB CLI
    Usuário: arthur_martins [PROFESSOR]
=====
Área do PROFESSOR em desenvolvimento.
1. Fazer Logout (Encerrar Sessão)
Escolha uma opção: █

```

```

=====
                ClassroomPB CLI
    Usuário: arthur_martins [PROFESSOR]
=====
Área do PROFESSOR em desenvolvimento.
1. Fazer Logout (Encerrar Sessão)
Escolha uma opção: █

```

US03 — Cadastro de Cursos no Sistema e Bloqueio de Código Duplicado

- **Descrição Técnica:** Implementação do fluxo de registro da estrutura acadêmica base na camada de negócios (**CursoService**). O sistema realiza o roteamento de privilégios interceptando o perfil do usuário ativo e exigindo direitos restritos a administradores. A camada de persistência local (**CursoRepository**) efetua buscas lineares para validar o critério de unicidade, bloqueando o salvamento físico caso o código identificador já exista na base de dados plano. Mensagens tratadas são propagadas à camada de apresentação (**AdminCursoCLI**) via interceptação de exceções customizadas (**ValidacaoException**), isolando saídas limpas no terminal.

```

=====
                ClassroomPB CLI
Usuário: Jose_Rafael [ADMINISTRADOR]
=====
1. Cadastrar Período Letivo
2. Ativar/Iniciar Período Letivo
3. Encerrar Período Letivo
4. Cadastrar Novo Curso
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 4
Código do Curso (ex: CC): EE
Nome Completo do Curso: Enfermagem
Sucesso: Curso 'Enfermagem' registrado na base de dados.

=====
                ClassroomPB CLI
Usuário: Jose_Rafael [ADMINISTRADOR]
=====
1. Cadastrar Período Letivo
2. Ativar/Iniciar Período Letivo
3. Encerrar Período Letivo
4. Cadastrar Novo Curso
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 4
Código do Curso (ex: CC): EE
Nome Completo do Curso: Enfermagem
ERRO DE NEGÓCIO: Erro: Já existe um curso cadastrado com o código 'EE'.

```

US04 — Cadastro de Disciplinas e Vinculação de Pré-requisitos Opcionais

- **Descrição Técnica:** Modelagem e persistência modular da grade curricular por meio da entidade **Disciplina**. O motor de validação na camada de negócios (**DisciplinaService**) restringe a operação ao perfil de coordenador e impõe regras estritas de integridade sobre os parâmetros quantitativos, exigindo valores inteiros estritamente positivos para carga horária e créditos. Adicionalmente, o sistema implementa uma validação de integridade referencial cruzada que inspeciona a base de dados local para garantir que quaisquer códigos informados como pré-requisitos opcionais

correspondam a disciplinas previamente homologadas no sistema, impedindo também o acoplamento cíclico (autovinculação).

```
=====
                        ClassroomPB CLI
Usuário: Joao [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma
3. Editar Turma Existente
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 1
Código da Disciplina (ex: P1): P5
Nome da Disciplina: Probabilidade
Carga Horária (inteiro positivo): 60
Créditos (inteiro positivo): 2
Possui código de pré-requisito? (Deixe em branco se não):
Sucesso: Disciplina 'Probabilidade' cadastrada com êxito!
```

US05 — Cadastro e Controle de Estados do Período Letivo

- **Descrição Técnica:** Modelagem do ciclo de vida temporal da aplicação por meio da entidade **Período**. Os estados são isolados logicamente em categorias controladas (**PLANEJADO**, **INICIADO**, **ENCERRADO**). Foi injetada uma regra de validação que bloqueia a ativação de qualquer novo período caso o sistema detecte que já existe outro período letivo em andamento (status **INICIADO**), garantindo a consistência cronológica dos dados.

```
=====
                        ClassroomPB CLI
Usuário: Keison_Raniery [ADMINISTRADOR]
=====
1. Cadastrar Período Letivo
2. Ativar/Iniciar Período Letivo
3. Encerrar Período Letivo
4. Cadastrar Novo Curso
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 1
Digite o código do período (ex: 2026.1): 2027.1
ERRO DE NEGÓCIO: Já possui um periodo (2026.1) cadastrado no sistema.
```

US06 — Oferta de Turmas para Disciplinas em Períodos Ativos

- **Descrição Técnica:** Acoplamento relacional básico e validação de integridade referencial na camada de negócios. O sistema impede a criação física de uma oferta de turma caso a disciplina associada não esteja previamente cadastrada no sistema ou se o período letivo informado pelo operador não estiver com o status marcado estritamente como **INICIADO**.

```

Usuário: alessia_bianca [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma (US10/US11)
3. Editar Turma Existente (US13/US14)
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: 00000
ID/Matrícula do Professor: 0003
Período Letivo (ex: 2026.1): 2026.1
Limite de Vagas: 40
Horário (ex: 08:00-10:00): 08:00-10:00
Sala (ex: Sala B3): sala_b4
ERRO DE VALIDACAO: Acao bloqueada: Nao eh possivel ofertar uma turma para uma disciplina inexistente.

```

```

Usuário: alessia_bianca [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma (US10/US11)
3. Editar Turma Existente (US13/US14)
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: 00000
ID/Matrícula do Professor: 0003
Período Letivo (ex: 2026.1): 2026.1
Limite de Vagas: 40
Horário (ex: 08:00-10:00): 08:00-10:00
Sala (ex: Sala B3): sala_b4
ERRO DE VALIDACAO: Acao bloqueada: Nao eh possivel ofertar uma turma para uma disciplina inexistente.

```

US09 — Controle de Ativação e Encerramento de Período Letivo

- **Descrição Técnica:** Desenvolvimento das operações core de transição de estado definitivo no ciclo de vida acadêmico (**PeriodoService**). O método de ativação efetua a mudança de estado para **INICIADO**, respeitando a restrição de unicidade. O método de encerramento faz a transição reversível e permanente para **ENCERRADO**. Uma vez encerrado, o período é blindado pelo sistema, bloqueando qualquer inserção posterior de novas turmas ou movimentações de alunos.

```

Usuário: alessia_bianca [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma (US10/US11)
3. Editar Turma Existente (US13/US14)
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: P1
ID/Matrícula do Professor: 0003
Período Letivo (ex: 2026.1): 2026.1
Limite de Vagas: 40
Horário (ex: 08:00-10:00): 08:00-10:00
Sala (ex: Sala_B3): sala_b4
ERRO DE VALIDAÇÃO: Ação bloqueada: O periodo letivo '2026.1' nao esta ativo (Status Hudson atual: ENCERRADO).

```

US10 — Oferta de Turmas para Disciplinas pelo Coordenador

- Descrição Técnica:** Disponibilização dos canais de entrada de texto e roteamento de privilégios para a montagem e publicação da grade horária oficial. A interface de linha de comando ([CoordenadorCLI](#) / [AdminCLI](#)) efetua o parsing textual do comando de oferta pública, tratando os parâmetros e repassando o fluxo para os serviços de persistência expostos aos alunos.

```

=====
ClassRoomPB CLI
Usuário: arthur_martins [PROFESSOR]
=====
Área do PROFESSOR em desenvolvimento.
1. Fazer Logout (Encerrar Sessão)
Escolha uma opção: █
ava: Ready

```

```

=====
ClassRoomPB CLI
Usuário: arthur_martins [PROFESSOR]
=====
Área do PROFESSOR em desenvolvimento.
1. Fazer Logout (Encerrar Sessão)
Escolha uma opção: █
ava: Ready

```



```
=====
ClassRoomPB CLI
Usuário: Keison_Raniery [ADMINISTRADOR]
=====
1. Cadastrar Período Letivo
2. Ativar/Iniciar Período Letivo
3. Encerrar Período Letivo
4. Cadastrar Novo Curso
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: █
```

(O menu adaptativo da TerminalCLI não exibe a opção de "Ofertar Nova Turma" para estes usuários(fornece apenas pra "Coordenador"), provando que o roteamento de privilégios está ativo.)

```
=====
ClassRoomPB CLI
Usuário: alessia_bianca [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma (US10/US11)
3. Editar Turma Existente (US13/US14)
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: P1
ID/Matrícula do Professor: 0003
Período Letivo (ex: 2026.1): 2027.1
Limite de Vagas: 40
Horário (ex: 08:00-10:00): 09:00-11:00
Sala (ex: Sala_B3): Sala_C104
SUCESSO: Turma ofertada com sucesso!
```

US11 — Definição de Atributos, Restrições e Responsáveis da Turma

- **Descrição Técnica:** Implementação do motor algorítmico de consistência física, restrições e alocação de recursos das turmas ([TurmaService](#)). O sistema exige obrigatoriamente a passagem de parâmetros técnicos de controle: Professor Responsável, Limite de Vagas (validado como número inteiro positivo), Horário e Sala de Aula. O motor realiza varreduras completas na base de dados simulando colisões espaciais e temporais, disparando as exceções de negócio [ChoqueHorarioException](#) ou

ChoqueSalaException para impedir o agendamento duplo de um mesmo docente ou da mesma sala no mesmo horário.

```
=====  
ClassRoomPB CLI  
Usuário: alessia_bianca [COORDENADOR]  
=====  
1. Cadastrar Nova Disciplina  
2. Ofertar Nova Turma (US10/US11)  
3. Editar Turma Existente (US13/US14)  
4. Cancelar Oferta de Turma  
5. Fazer Logout (Encerrar Sessão)  
=====  
Escolha uma opção: 2  
ID da Disciplina: P1  
ID/Matrícula do Professor: 0003  
Período Letivo (ex: 2026.1): 2027.1  
Limite de Vagas: 40  
Horário (ex: 08:00-10:00): 07:00-09:00  
Sala (ex: Sala B3): Sala_C104  
[CONFLITO DE HORÁRIO] Erro de Conflito: O professor '0003' já está alocado na disciplina 'P1' neste mesmo período e horário (07:00-09:00).
```

(Choque de horário)

```
=====  
ClassRoomPB CLI  
Usuário: alessia_bianca [COORDENADOR]  
=====  
1. Cadastrar Nova Disciplina  
2. Ofertar Nova Turma (US10/US11)  
3. Editar Turma Existente (US13/US14)  
4. Cancelar Oferta de Turma  
5. Fazer Logout (Encerrar Sessão)  
=====  
Escolha uma opção: 2  
ID da Disciplina: P1  
ID/Matrícula do Professor: 0003  
Período Letivo (ex: 2026.1): 2027.1  
Limite de Vagas: -10  
Horário (ex: 08:00-10:00): 08:00-10:00  
Sala (ex: Sala_B3): sala_b4  
ERRO DE VALIDACAO: Acao bloqueada: O limite de vagas deve ser um inteiro positivo.
```

(Limite de vagas obrigatoriamente tem que ser um número inteiro e positivo)

US12 — Validação de Conflito de Horário de Professor

- **Descrição Técnica:** Desenvolvimento do motor algorítmico de consistência temporal integrado ao pipeline de publicação de turmas (**TurmaService**). Antes de autorizar a persistência de um novo registro, o sistema realiza uma varredura completa na base de dados local (**turmas.txt**) filtrando as alocações pelo período letivo vigente e pela matrícula do professor responsável. O motor analisa colisões de strings na propriedade de horário e, caso detecte o agendamento duplo de um mesmo docente no mesmo intervalo de tempo, aborta a operação disparando a exceção de negócio **ChoqueHorarioException**, a qual é capturada pela interface de linha de

comando (**CoordenadorCLI** / **AdminCLI**) para a exibição de alertas de conflito em tempo de execução.

```
=====
ClassRoomPB CLI
Usuário: Joao [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma
3. Editar Turma Existente
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: P6
ID/Matrícula do Professor: 1234
Período Letivo (ex: 2026.1): 2026.1
Limite de Vagas: 55
Horário (ex: 08:00-10:00): 11
Sala (ex: Sala_B3): Sala_101
SUCESSO: Turma ofertada com sucesso!

=====
ClassRoomPB CLI
Usuário: Joao [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma
3. Editar Turma Existente
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: P5
ID/Matrícula do Professor: 1234
Período Letivo (ex: 2026.1): 2026.1
Limite de Vagas: 55
Horário (ex: 08:00-10:00): 11
Sala (ex: Sala_B3): Sala_102
[CONFLITO DE HORÁRIO] Erro de Conflito: O professor '1234' já está alocado na disciplina 'P6' neste mesmo período e horário (11).
```

US13 — Obrigatoriedade de Professor Responsável na Turma

- **Descrição Técnica:** Injeção de cláusulas de barreira e travas de consistência física aplicadas de forma simétrica nos métodos de criação (**ofertarTurma**) e mutação de estado (**editarTurma**) na camada de serviço. O sistema valida de maneira estrita a presença da referência ao docente, interceptando e rejeitando qualquer cadeia de caracteres nula, vazia ou composta exclusivamente por espaços latentes na propriedade de matrícula do professor. A restrição garante a integridade referencial contínua da persistência local, impossibilitando que uma turma exista ou seja alterada no arquivo plano sem um docente associado, disparando **IllegalArgumentException** imediatamente em caso de violação.

US14 — Gestão de Alteração e Cancelamento de Turmas (Sprint 2)

- **Descrição Técnica:** Injeção de cláusulas de barreira e travas de consistência física aplicadas de forma simétrica nos métodos de mutação de estado (**editarTurma**) e remoção (**cancelarTurma**) na camada de serviço (**TurmaService**). O sistema valida de maneira estrita o status do período letivo associado por meio do método **validarStatusPeriodo**, abortando a

operação e disparando `IllegalStateException` se o período estiver `INICIADO` ou `ENCERRADO`. A exclusão física e lógica é realizada em memória e consolidada em disco através do método `atualizarArquivoCompleto` do `TurmaRepository`, que reescreve o arquivo plano de armazenamento de forma limpa.

```
=====
                ClassroomPB CLI
Usuário: Joao [COORDENADOR]
=====
1. Cadastrar Nova Disciplina
2. Ofertar Nova Turma
3. Editar Turma Existente
4. Cancelar Oferta de Turma
5. Fazer Logout (Encerrar Sessão)
=====
Escolha uma opção: 2
ID da Disciplina: P5
ID/Matrícula do Professor:
Período Letivo (ex: 2026.1): 2026.1
Limite de Vagas: 55
Horário (ex: 08:00-10:00): 12
Sala (ex: Sala_B3): sala_11
ERRO DE VALIDAÇÃO: Ação bloqueada: Nao eh possivel ofertar uma turma sem um professor responsavel.
```

5. Relatório de Testes Automatizados e Cobertura

Para mitigar a introdução de regressões no ClassroomPB e atender às exigências de rigor técnico, foi implementada uma suíte automatizada com o framework JUnit 4. Os testes simulam exaustivamente tanto caminhos felizes de fluxo de dados quanto fluxos de exceção severos.

Cenários Cobertos:

1. `testCadastroSucessoEPerfil`: Valida a correta instânciação e classificação do perfil inserido via fábrica.
2. `testImpedimentoDuplicidadeMatricula / testImpedimentoDuplicidadeEmail`: Garante o isolamento de dados e o disparo da exceção de conflito de cadastro.
3. `testNegacaoDeAcessoParaSenhaIncorreta`: Garante que tentativas de invasão com credenciais inválidas bloqueiem a geração de sessão.

4. **testLoginComDadosNulosEVazios**: Trata e intercepta argumentos nulos ou em branco fornecidos na interface.
5. **testConflitoDeHorarioESala**: Valida que o motor antichoques impeça a persistência de turmas inválidas, assegurando a integridade das restrições do mundo real.
6. **deveCadastrarCursoComSucesso**: Garante que um Administrador consegue cadastrar um curso válido (como "CC") e que os dados são salvos corretamente na persistência.
7. **deveLancarExcecaoQuandoUsuarioNaoForAdmin**: Bloqueia o cadastro se o usuário ativo não for um Administrador (ex: Aluno tentando cadastrar curso).
8. **deveLancarExcecaoQuandoCodigoForDuplicado**: Impede que dois cursos diferentes usem o mesmo código identificador.
9. **deveLancarExcecaoQuandoCamposForemInvalidos**: Valida a obrigatoriedade dos dados, rejeitando códigos ou nomes deixados em branco.
10. **deveCadastrarDisciplinaComSucesso**: Valida se um Coordenador consegue cadastrar uma disciplina básica sem problemas.
11. **deveCadastrarComPreRequisitosValidos**: Confirma que o sistema aceita vincular pré-requisitos reais (outras disciplinas que já existem no arquivo).
12. **deveLancarExcecaoQuandoUsuarioNaoForCoordenador**: Protege a rota garantindo que apenas Coordenadores tenham permissão para criar disciplinas.
13. **deveLancarExcecaoParaValoresNegativosOuZerar**: Barra cadastros se a carga horária for zero ou se a quantidade de créditos for um número negativo.
14. **deveLancarExcecaoParaCodigoDuplicado**: Impede o salvamento de uma disciplina com o mesmo código de outra já existente.
15. **deveLancarExcecaoSePreRequisitoNaoExistir**: Bloqueia o registro se o Coordenador tentar vincular uma disciplina fantasma (inexistente) como pré-requisito.
16. **deveOfertarTurmaComSucessoQuandoPeriodoAtivoEDisciplinaExistente**: Valida o fluxo perfeito: Coordenador alocando turma para disciplina real em período letivo iniciado.
17. **deveLancarExcecaoQuandoUsuarioNaoForCoordenador**: Certifica que perfis sem permissão não consigam publicar ofertas de turmas.

18. **deveLancarExcecaoQuandoDisciplinaNaoExistirNoSistema:** Impede a criação de turmas vinculadas a códigos de disciplinas inexistentes.
19. **deveLancarExcecaoQuandoPeriodoNaoEstiverAtivo:** Garante que nenhuma turma seja aberta em períodos ainda em fase de planejamento.
20. **deveLancarExcecaoQuandoProfessorJaTemTurmaNoMesmoHorarioEPeriodo:** Testa o motor antichoque, impedindo que um docente seja colocado em duas salas diferentes no mesmo horário.
21. **deveLancarExcecaoQuandoHouverChoqueDeSalaNoMesmoHorarioEPeriodo:** Evita colisões de espaço físico (duas turmas na mesma sala de aula no mesmo horário).
22. **devePermitirMesmaSalaEMesmoHorarioSeOsPeriodosLetivosForemDiferentes:** Confirma que a mesma sala de aula pode ser reutilizada no mesmo horário se for em semestres/períodos diferentes.
23. **deveLancarExcecaoQuandoAtributosObrigatoriosContiveremApenasEspacos:** Rejeita parâmetros essenciais (como horário ou sala) preenchidos apenas com espaços em branco.
24. **deveLancarExcecaoQuandoLimiteDeVagasForInvalido:** Exige que a quantidade de vagas da turma seja um número inteiro positivo (maior que zero).
25. **deveLancarExcecaoQuandoCamposForemNulos:** Evita falhas no sistema barando qualquer propriedade da turma enviada como nula.
26. **deveLancarExcecaoQuandoProfessorForVazio: (US13)** Impede que uma turma seja criada sem possuir a matrícula de um docente associado.
27. **deveEditarTurmaComSucessoQuandoPeriodoEstiverPlanejado:** Permite alterações cadastrais e de professor responsável se o período ainda não tiver iniciado.
28. **deveLancarExcecaoAoTentarEditarTurmaDeixandoProfessorNuloOuVazio:** Assegura que, mesmo durante uma edição, ninguém consiga remover o professor responsável e deixar a turma sem docente.
29. **deveCancelarTurmaComSucessoRemovendoDoRepositorio:** Confirma a exclusão física e lógica de uma oferta de disciplina da base de memória.
30. **deveImpedirEdicaoSePeriodoEstiverIniciado:** Protege a integridade do semestre letivo, trancando modificações na turma se o período já estiver em andamento.

31. **deveImpedirCancelamentoSePeriodoEstiverEncerrado:**

Blindagem histórica: impede a exclusão ou alteração de turmas de períodos que já foram finalizados e consolidados.

Indicadores de Cobertura:

***[INSERIR AQUI PRINT DO RELATÓRIO DE COBERTURA (Jacoco ou Aba Test do VS Code) DEMONSTRANDO COBERTURA ACIMA DE 80%]**